



BITS Pilani
DIGITAL

Excellence made yours

Vision-Based Lane Inference for Unstructured and Structured Indian Roads

Presented by:

**Harshwardhan Mukund
Mohadikar • Shreyas Bhat K**

BSc Computer Science • 2025-26

Under the guidance of
Dr. Ashok Yemineni

Problem Statement

Lane detection assumes paint. Most of the Indian network has none.

What every method assumes

THE SHARED PREMISE

- Painted lane markings are present in the frame
- SCNN and LaneNet label marking pixels, then cluster them
- Ultra Fast LDD picks which cell in a row holds a marking
- All trained on TuSimple, CULane and BDD100K

What Indian roads give you

THE ACTUAL INPUT

- Paint is absent, worn, or hidden under traffic
- Yet the road still carries a structure drivers agree on
- That structure is in the geometry, not the markings

And the failure is silent

A detector that finds nothing looks exactly like one that has failed. Nothing downstream can tell them apart, so on an unmarked road the honest output is a refusal, not a guess.

Objectives & Scope

Rebuild the pipeline so every claim it makes can be measured.

Primary objectives

WHAT TO BUILD

- Review lane detection and inference for Indian road settings
- Infer lanes on structured, semi-structured and unstructured roads, from geometry rather than markings
- Make that model optimised, effective and lightweight enough for video rate on commodity hardware
- Validate against ground truth and trivial baselines, with a confidence that predicts its own error

Secondary objectives

ATTEMPTED, OUTCOMES REPORTED

- Wrong-side driving from inferred lane direction: implemented, unvalidated
- Traffic flow in unorganised conditions: measured and rejected
- Feasibility for basic driver assistance: the limitations analysis

Scope

BOUNDARIES

- In: monocular forward-facing camera
- In: single image, video clip, live camera
- In: lane count and metric widths, both validated
- Also reported, not validated: ego lane, road-user distances
- Out: path planning, vehicle control, any other sensor
- Out: night and heavy rain

METHODS THIS RESTS ON

MobileNetV3

LR-ASPP

FPN

IPM

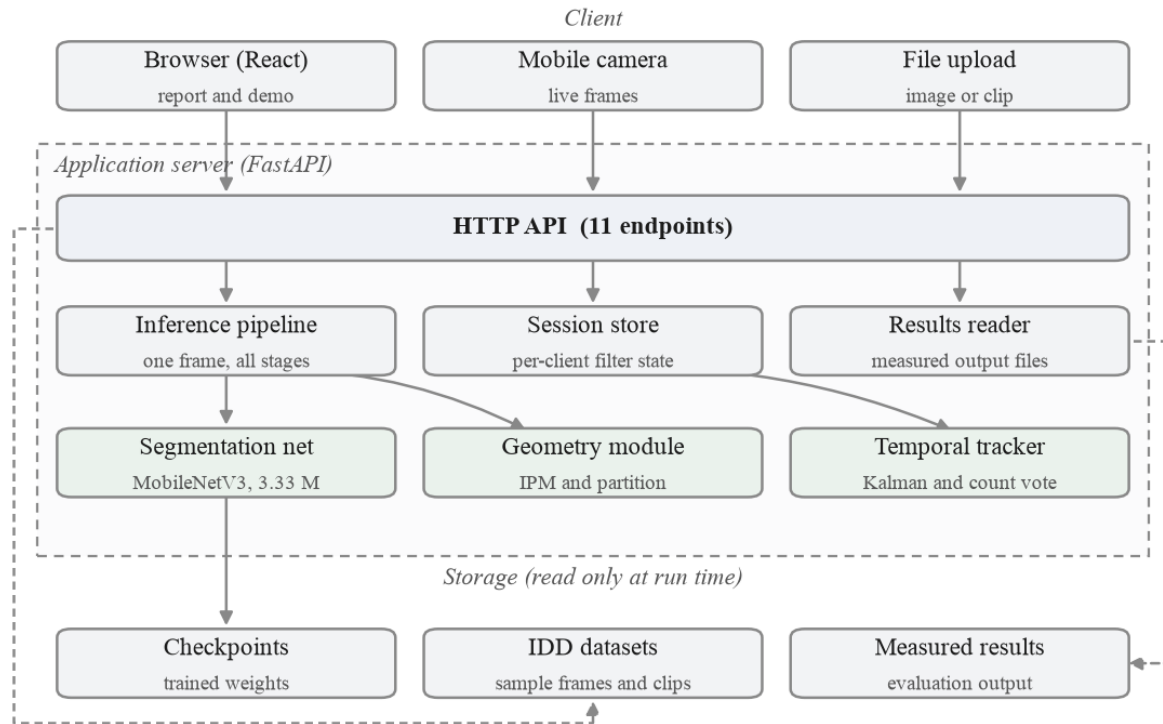
Kalman filter

IRC:86

Knowledge distillation

System Architecture

Four tiers, one API surface, and everything in the dashed box is one process.



React interface

FastAPI, 11 endpoints

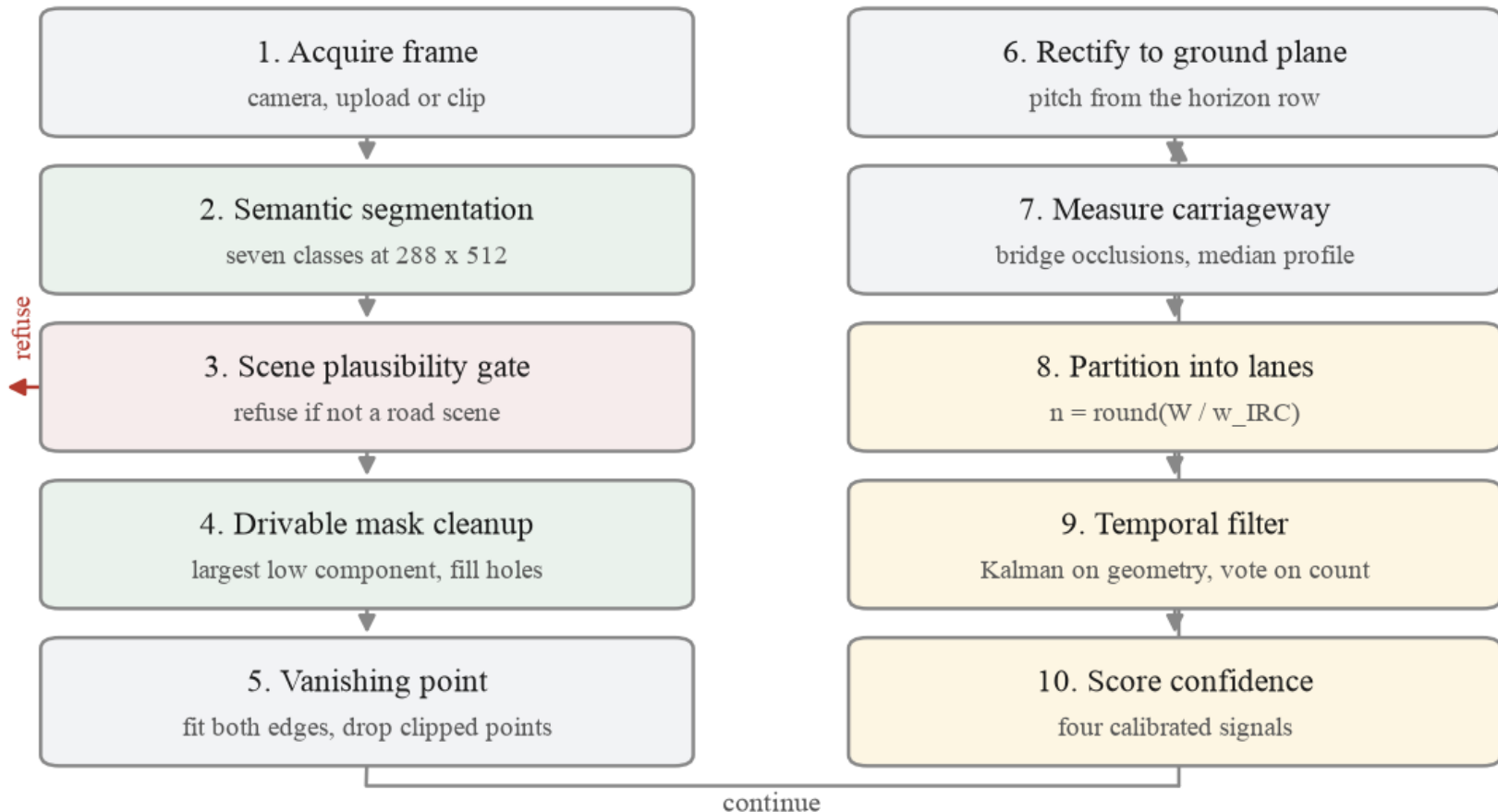
Stateless inference

Read-only weights and results

End-to-End Process Flow

Ten stages, including the path where the system declines to answer.

Every stage writes an intermediate image, which is what the stage walkthrough in the demo displays.



Focal Length Cancels

The dataset ships no camera metadata. It turns out not to need it.

The problem

NO CALIBRATION

- Metric output normally needs the focal length
- Arbitrary dashcam footage does not carry it
- Pitch is measured per frame from the vanishing point

The algebra

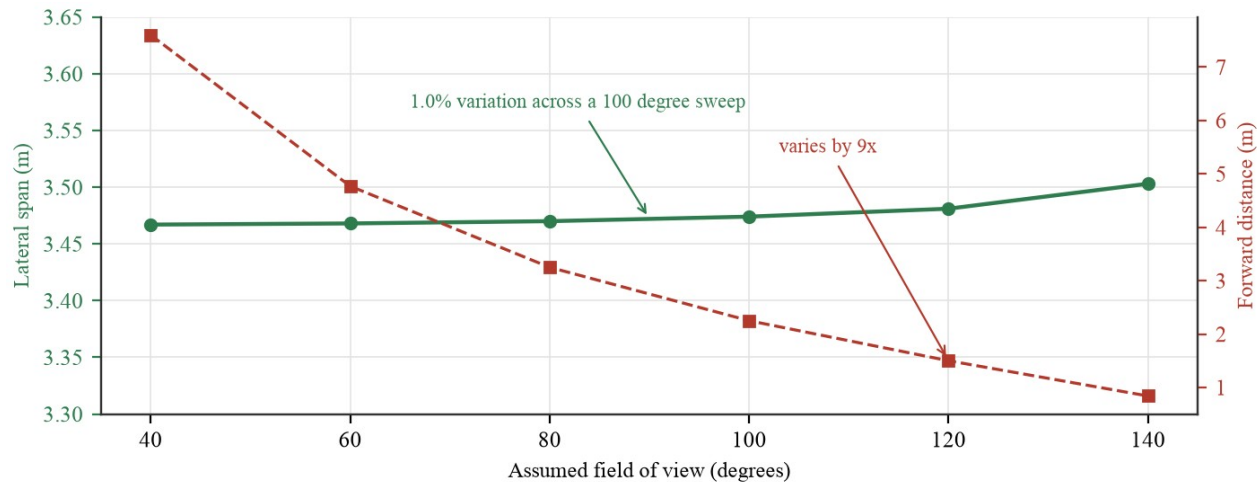
F DIVIDES OUT

- $dX = du (f H / (v - v_h)) / f$
- which is $du H / (v - v_h)$
- f appears above and below, and cancels

The evidence

MEASURED, NOT ASSERTED

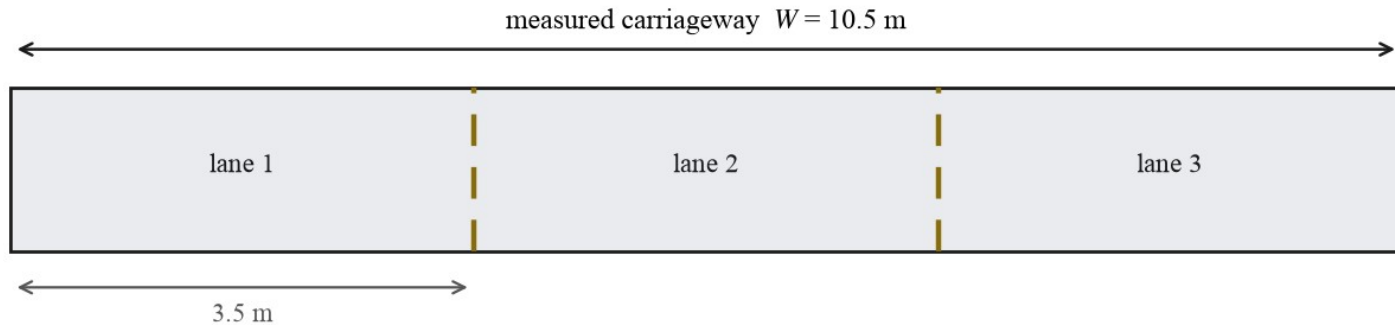
- Field of view swept 40 to 140 degrees
- Lateral scale moves 1.0 per cent
- Against a 11.8 per cent median width error



From Width to Lane Count

A published design width applied to a measurement, not an arbitrary divisor.

The count follows a measured width divided by a design standard, not a fixed subdivision.



$$n = \text{round}(W/w_{\text{IRC}}) = \text{round}(10.5/3.5) = 3$$

The rule

$$N = \text{ROUND}(W / w)$$

- $w = 3.50$ m at or above a 6.0 m carriageway
- $w = 3.00$ m below it, a narrower road
- A 2.5 m floor rejects a sliver of visible tarmac

Why a threshold, not a classifier

DELIBERATE

- No scene is labelled urban or rural anywhere
- The threshold is on the measured width itself
- Nothing has to be recognised for it to hold

The Lane-Marking Head

IDD has no lane annotation, so YOLOP was used offline as a teacher.

input frame



lane-line probability



thresholded at 0.5



marked road: 6.42% of pixels above threshold



unmarked road: 0.00% of pixels above threshold

Distillation

1,607 PSEUDO-LABELS

- YOLOP labelled IDD offline, never at run time
- A second head trained against those labels
- Shares one forward pass with segmentation

What it cost, and bought

BOTH REPORTED

- 0.011 mIoU on the semantic task
- Structured frames rose from 27 to 39
- YOLOP is 150 ms on CPU; our head is free

Implementation

A running application, not a set of scripts. Everything pinned and reproducible.

Model

3.33 M PARAMETERS

- MobileNetV3-Large backbone, LR-ASPP head, FPN decoder
- Seven semantic classes plus the distilled lane head
- IDD-20k: 6,993 train, 981 val, 40 epochs, AdamW

Pipeline

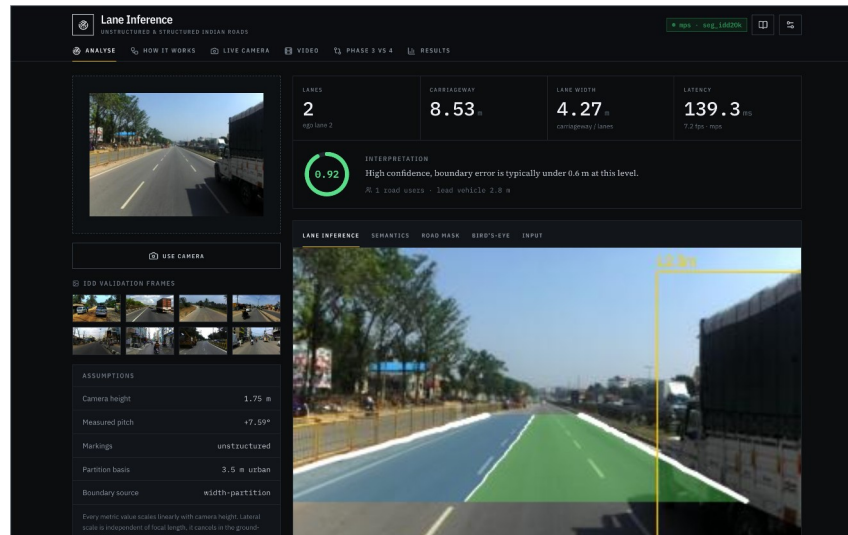
GEOMETRY AND TRACKING

- Vanishing point, inverse perspective mapping
- Per-row occlusion handling in the width profile
- Kalman filter over centreline and width, count vote

Delivery

SERVED AND TESTED

- FastAPI, eleven endpoints; React interface
- ONNX Runtime: 226 MB resident, no PyTorch
- 58 tests over geometry, metrics, API and export



Pipeline Stages

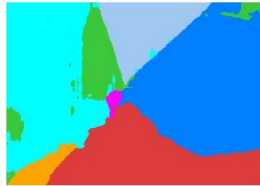
A narrow road, a two-lane road, a painted three-lane road, and a refusal.

Input frame



1 lane(s), 3.25 m carriageway, confidence 0.92, 112 ms

Semantic segmentation



Drivable mask, cleaned



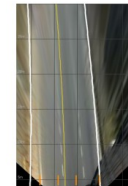
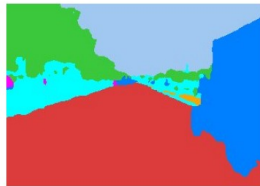
Metric bird's-eye view



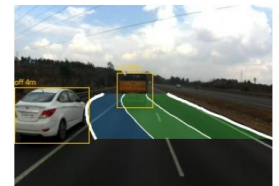
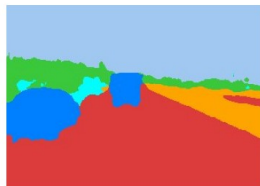
Inferred lane boundaries



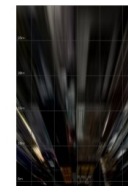
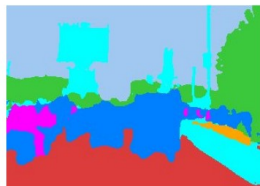
2 lane(s), 8.53 m carriageway, confidence 0.92, 44 ms



3 lane(s), 9.61 m carriageway, confidence 0.90, 41 ms



no lane solution (road scene)



Results & Analysis

Measured on the 204-frame IDD-Lite validation split, by a harness that ships with the code.



0.9310

drivable IoU, deployed

0.7754

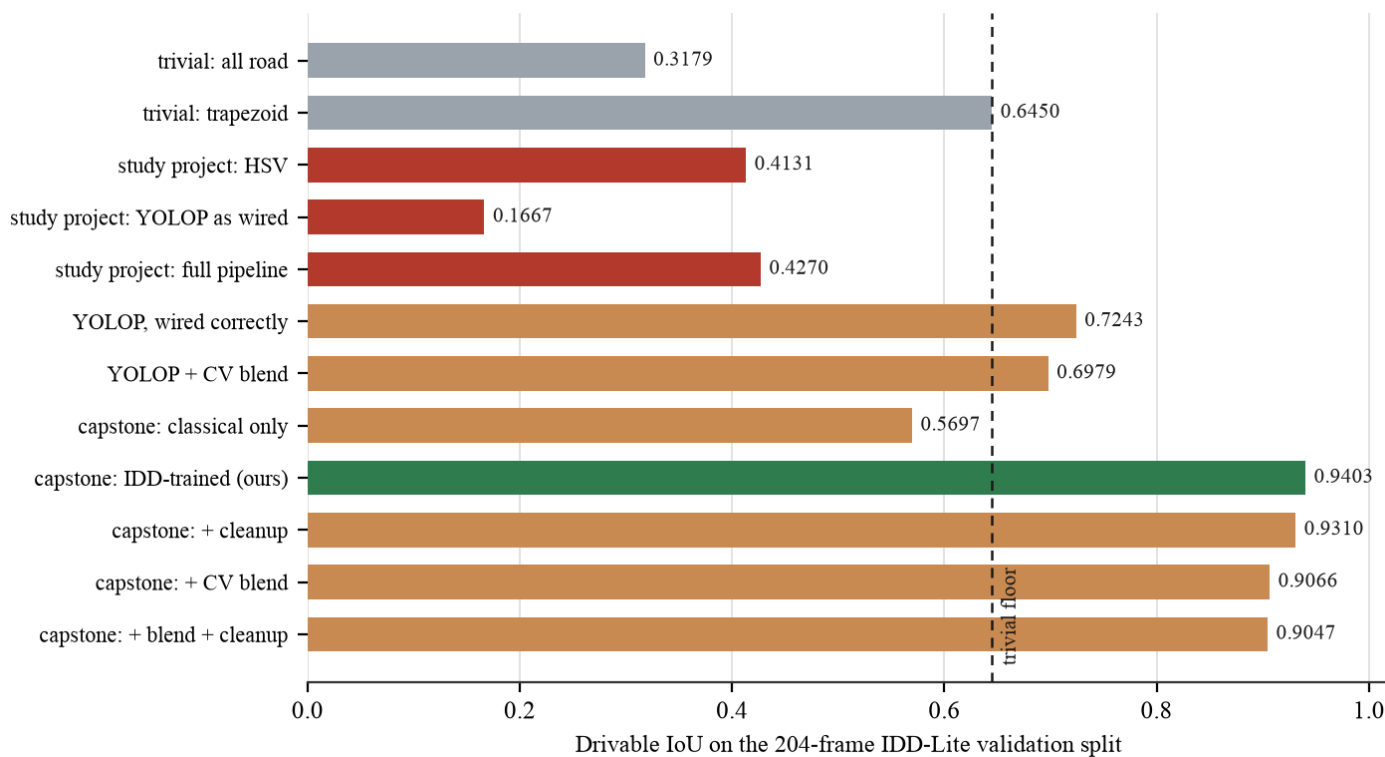
boundary F1, deployed

165/204

lane solutions

27 fps

end to end, 36.5 ms



Is the Geometry Correct?

The hardest question in the project, and the one an examiner should ask.

0.314 m

median road-edge error

11.8%

median width error

39

refused of 204, by design

4.3

lane flips per 100 frames

Against ground truth

2,288 SAMPLES

- Outermost inferred boundary against the drivable label
- 0.314 m median, 1.526 m at p90
- Interior divisions have no ground truth anywhere

Confidence that predicts error

CALIBRATED

- Built from signals measured to predict error
- Gating at 0.8 takes the median from 0.425 to 0.339 m
- So the tail is identifiable in advance

Stable across video

150 CLIPS, 4,526 FRAMES

- Width jitter 0.311 to 0.038 m
- Count flips 20.1 to 4.3 per 100
- Solution rate 80.4 to 85.4 per cent

Challenges & Limitations

Stated here rather than left for the viva to find.

What did not work

IMPLEMENTED, MEASURED, REJECTED

- Fusing the classical and learned branches: 0.9403 to 0.9066
- Bridging occlusions before the vanishing point
- Inferring lanes from observed traffic
- Four test-time remedies for the dashcam domain gap

What we cannot claim

LIMITATIONS

- Lane counts are inferred, not detected, and unvalidated
- Metric scale rests on an assumed 1.75 m camera height
- Ego lane and road-user distance are never evaluated
- Half the annotated data, IDD-20k Part II, is unused
- Not validated for control or any safety function

The defect that taught us the most

Bridging every vehicle gap joined our carriageway to the opposing one across a median. Found by checking the model against the mask it was built from, not against ground truth: the mask was within 0.20 m, the model 0.58 m wider. Handling occlusion per row took the p90 width error from 151 to 47 per cent.

Conclusion & Future Work

The idea holds, and it is measurable rather than plausible.

165/204

frames solved

0.31 m

median edge error

0.9403

drivable IoU

58

tests, all passing

What was shown

CONCLUSION

- Lane structure can be inferred without markings
- Metric scale needs no focal length, only a height
- A system that refuses is more useful than one that guesses

What is next

FUTURE WORK

- Merge IDD-20k Part II, half the annotated data
- Validate ego-lane assignment, the first unmeasured output
- Confirm the narrow-road divisor against printed IRC
- Close the consumer dashcam gap with training data

Thank you

Questions. · github.com/Arkanobot/VisionBasedLaneInference